

```
# Pandas -> Pandas is a python library which is used for the data
# manipulation and for the data analysis also...
# 1. Series
# 2. DataFrame
```

```
!pip install pandas
```

```
import pandas as pd
import numpy as np

# Series is a data type in pandas which is having one dimension and each
# of the elements have the labels.. that is known as series
arr = np.array([1.0,2,3.9,4.5])
# Series
s1 = pd.Series(arr, index=['first', 'second', 'third', 'fourth'])
s1.dtype
```

```
# Dataframe
import numpy as np
data = {
    'Name':['Ram', 'Shyam', 'Aman', 'Naman'],
    'Marks':[20, 30, 40, np.nan],
    'Location':['Jaipur', 'Delhi', 'Noida', 'LKO']
}
df = pd.DataFrame(data, index=['first', 'second', 'third', 'fourth'])
```

```
# To perform any string operations on the object data type column
# we use .str -> any function
# To add any new column into the dataframe then we use
# dataframe_name[new_column] = new_value
df['Upper_letters']=df['Location'].str.upper()
```

```
df['Marks'].max()
```

```
df['Marks'].mean()
```

```
df['Marks'].median()
```

```
df['Marks'].std()
```

```
df['Marks'].min()
```

```
df
```

```
# To check any null value from the series
# df['col'].isna() -> this returns the boolean value
# if the entry is true -> then it means there is null value present
# if the entry is false -> then there is not null is is present

# df['Marks'].isna()
```

```
df['Marks'].isna().sum()
```

```
# Create a dataframe using a dictionary
# Employee -> Name, Salary, YOE, Increment
# Atleast have one or more than one null value in each of the col
# first check the data type of each of the column seperately..
# check the highest salary of the person
# median salary..
# mode of the salary...
# how many people are the at the same experienc..
# check each of the column which has how many null values
# (null value count )
```

```
import pandas as pd

# Read dataset using pd.read_csv(csv_path)
df = pd.read_csv('/content/sample_data/california_housing_test.csv')
```

```
# Head method shows the first five rows of the dataset..
df.head()
```

```
# Tail method shows the last five rows of the dataset..
df.tail()
```

```
# Sample method shows the random rows of the dataset and by
# default it returns the 10 sample rows.
df.sample(10)
```

```
# .columns shows all the list of the columns which are presnt into the df
df.columns
```

```
df
```

```
# .info() it tells all the non values of the dataframe and
# and tells the data type of each of the columns
df.info()
```

```
# Decribe method tells the stastical property of the dataset..
# It also gives the the five point summary
# count -> total number of entries
# mean -> average of each of the numerical column
# std -> standard deviation
# 1. minimum value -> each of the columns
# 2. 25% -> 25th percentile
# 3. 50% -> Median value
# 4. 75% percnetile
# 5. Maximum value
df.describe()
```

```
df.shape
```

```
# for selecting the single column of the dataset or the dataframe
# dataframe['column_name']
df['new_col'] = df['total_rooms'].mean()
```

```
import pandas as pd

data = {
    'Math': [80, 90, 70],
    'Science': [85, 95, 75]
}
df = pd.DataFrame(data, index=['Alice', 'Bob', 'Charlie'])
```

```
df['Total']=df.apply(lambda data: data['Math']+data['Science'], axis=1)
```

```
df
```

	Math	Science	Total
Alice	80	85	165
Bob	90	95	185
Charlie	70	75	145

```
df
```

	Math	Science
Alice	80	85
Bob	90	95
Charlie	70	75

```
df.mean(axis=1)
```

	0
Alice	82.5
Bob	92.5
Charlie	72.5

```
dtype: float64
```

```
df.mean(axis=0)
```

	0
Math	80.0
Science	85.0

```
dtype: float64
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```
# For selecting the multiple columns from the dataset we use the list of
# columns
# dataframe_name[[list of the columns]] -> new dataframe

df[['latitude', 'population']]
```

```
df
```

```
# df
```

```
# For selecting the particular rows and columns using iloc
# df.iloc[[list of the row], [list of columns]]
# df.iloc[1:11]
# df.iloc[1:11, 0:5]

df.iloc[[0,10, 11], [3, 5,6]]
```

```
# Iloc -> iloc is used for the integer based indexing.
```

```
df.iloc[1:11, 0:3]
```

```
# For selecting the particular dataframe from the whole dataset
# we can use the slicing into the
# df.iloc[row_slicing, column_slicing]
# Exclusive end..
```

```
import pandas as pd
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston'],
    'Marks':[33, 56, 76,65]
}

df = pd.DataFrame(data, index=['A', 'B', 'C', 'D'])
```

```
df[df['Age'] >= 30]
```

```
# For selecting the values of the dataframe on the basis of some condition
# df[column] condition
# df[expression]
```

	Name	Age	City	Marks
B	Bob	30	Los Angeles	56
C	Charlie	35	Chicago	76

```
df1 = df[df['Marks'] <= 75]
```

```
df1['bonus_marks'] = df1['Marks'] + 5
```

```
/tmp/ipykernel_15819/818156461.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-ver
df1['bonus_marks'] = df1['Marks'] + 5
```

```
df1
```

	Name	Age	City	Marks	bonus_marks
A	Alice	25	New York	33	38
B	Bob	30	Los Angeles	56	61
D	David	28	Houston	65	70

```
import pandas as pd
```

```
# Creating the dataset with the added Gender column
```

```
data = {
    'Student_ID': list(range(101, 116)),
    'Name': [
        'Alice', 'Bob', 'Charlie', 'David', 'Eva',
        'Frank', 'Grace', 'Henry', 'Ivy', 'Jack',
        'Karl', 'Liam', 'Mia', 'Noah', 'Olivia'
    ],
    'Age': [20, 21, 19, 22, 20, 23, 21, 22, 20, 19, 21, 22, 20, 21, 19],
    'Gender': [
        'Female', 'Male', 'Male', 'Male', 'Female',
        'Male', 'Female', 'Male', 'Female', 'Male',
        'Male', 'Male', 'Female', 'Male', 'Female'
    ],
    'Major': [
        'Computer Science', 'Mathematics', 'Physics', 'Biology', 'Chemistry',
        'Computer Science', 'Mathematics', 'Physics', 'Biology', 'Chemistry',
        'Computer Science', 'Mathematics', 'Physics', 'Biology', 'Chemistry'
    ],
    'GPA': [3.8, 3.2, 3.9, 2.8, 3.5, 3.6, 3.1, 3.7, 3.4, 2.9, 3.9, 3.3, 4.0, 3.0, 3.6]
}
```

```
df = pd.DataFrame(data)
```

```
# Display the updated DataFrame
print(df)
```

	Student_ID	Name	Age	Gender	Major	GPA
0	101	Alice	20	Female	Computer Science	3.8
1	102	Bob	21	Male	Mathematics	3.2
2	103	Charlie	19	Male	Physics	3.9
3	104	David	22	Male	Biology	2.8
4	105	Eva	20	Female	Chemistry	3.5
5	106	Frank	23	Male	Computer Science	3.6
6	107	Grace	21	Female	Mathematics	3.1
7	108	Henry	22	Male	Physics	3.7
8	109	Ivy	20	Female	Biology	3.4
9	110	Jack	19	Male	Chemistry	2.9
10	111	Karl	21	Male	Computer Science	3.9
11	112	Liam	22	Male	Mathematics	3.3

```

12      113      Mia      20  Female      Physics  4.0
13      114      Noah      21   Male      Biology  3.0
14      115     Olivia      19  Female     Chemistry 3.6

```

```
# Map function or the method is used to map all the values of the df[col]
```

```
df['Minor/Major'] = df['Age'].map(lambda age: 'Minor' if age <= 19 else 'Major')
```

```
# Unique method -> Unique method is used find out all the categories of the categorical
# column..
df['Major'].unique()
```

```
array(['Computer Science', 'Mathematics', 'Physics', 'Biology',
      'Chemistry'], dtype=object)
```

```
# Value counts method is used to find the number or counts of each of the category of
# particular column
df['Major'].value_counts()
```

```

              count
Major
Computer Science    3
Mathematics         3
Physics             3
Biology             3
Chemistry           3

```

```
dtype: int64
```

```
# Add a new column of Pass/Fail on basis given condition that if the student
# scores less 5gpa -> fail, otherwise pass
```

```

      Student_ID  Name  Age  Gender      Major  GPA
0           101  Alice   20  Female  Computer Science  3.8
1           102   Bob   21   Male    Mathematics  3.2
2           103  Charlie  19   Male      Physics  3.9
3           104   David  22   Male      Biology  2.8
4           105    Eva   20  Female     Chemistry  3.5
5           106   Frank  23   Male  Computer Science  3.6
6           107  Grace   21  Female    Mathematics  3.1
7           108  Henry   22   Male      Physics  3.7
8           109    Ivy   20  Female      Biology  3.4
9           110   Jack   19   Male     Chemistry  2.9
10          111   Karl   21   Male  Computer Science  3.9
11          112   Liam   22   Male    Mathematics  3.3
12          113    Mia   20  Female      Physics  4.0
13          114   Noah   21   Male      Biology  3.0
14          115  Olivia   19  Female     Chemistry  3.6

```

```
df['Gender'] = df['Gender'].map({'Female':0,'Male':1})
```

```
df['Major'].unique()
```

```
array(['Computer Science', 'Mathematics', 'Physics', 'Biology',
      'Chemistry'], dtype=object)
```

```

import pandas as pd
import numpy as np
# Dataset with separate subject columns

```

```

data = {
    'Student_ID': list(range(101, 116)),
    'Name': [
        'Alice', 'Bob', 'Charlie', 'David', 'Eva',
        'Frank', 'Grace', 'Henry', 'Ivy', 'Jack',
        'Karl', 'Liam', 'Mia', 'Noah', 'Olivia'
    ],
    'Age': [20, 21, 19, 22, 20, 23, 21, 22, 20, 19, 21, 22, 20, 21, 19],
    'Gender': [
        'Female', np.nan, 'Male', 'Male', 'Female',
        'Male', 'Female', 'Male', 'Female', 'Male',
        'Male', 'Male', np.nan, 'Male', 'Female'
    ],
    'Major': [
        'Computer Science', 'Mathematics', 'Physics', 'Biology', 'Chemistry',
        'Computer Science', 'Mathematics', 'Physics', 'Biology', 'Chemistry',
        'Computer Science', 'Mathematics', 'Physics', 'Biology', 'Chemistry'
    ],
    'Maths': [88, 92, 78, np.nan, 80, 95, np.nan, 74, 70, np.nan, 91, 85, 99, np.nan, 83],
    'Physics': [85, 80, 95, 70, 82, 88, 79, 91, 75, 66, 84, 87, np.nan, 72, 81],
    'Chemistry': [90, 78, np.nan, 80, 94, 82, 85, 88, 83, 76, 89, 80, 96, 74, 92]
}

df = pd.DataFrame(data)

# Display the updated DataFrame
print(df)

```

	Student_ID	Name	Age	Gender	Major	Maths	Physics	\
0	101	Alice	20	Female	Computer Science	88.0	85.0	
1	102	Bob	21	NaN	Mathematics	92.0	80.0	
2	103	Charlie	19	Male	Physics	78.0	95.0	
3	104	David	22	Male	Biology	NaN	70.0	
4	105	Eva	20	Female	Chemistry	80.0	82.0	
5	106	Frank	23	Male	Computer Science	95.0	88.0	
6	107	Grace	21	Female	Mathematics	NaN	79.0	
7	108	Henry	22	Male	Physics	74.0	91.0	
8	109	Ivy	20	Female	Biology	70.0	75.0	
9	110	Jack	19	Male	Chemistry	NaN	66.0	
10	111	Karl	21	Male	Computer Science	91.0	84.0	
11	112	Liam	22	Male	Mathematics	85.0	87.0	
12	113	Mia	20	NaN	Physics	99.0	NaN	
13	114	Noah	21	Male	Biology	NaN	72.0	
14	115	Olivia	19	Female	Chemistry	83.0	81.0	

	Chemistry
0	90.0
1	78.0
2	NaN
3	80.0
4	94.0
5	82.0
6	85.0
7	88.0
8	83.0
9	76.0
10	89.0
11	80.0
12	96.0
13	74.0
14	92.0

```
df = pd.read_csv('/content/breast-cancer.csv')
```

```
df.columns
```

```

Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')

```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                     569 non-null    int64
1   diagnosis                             569 non-null    object
2   radius_mean                           569 non-null    float64
3   texture_mean                           569 non-null    float64
4   perimeter_mean                         569 non-null    float64
5   area_mean                             569 non-null    float64
6   smoothness_mean                       569 non-null    float64
7   compactness_mean                      569 non-null    float64
8   concavity_mean                        569 non-null    float64
9   concave points_mean                   569 non-null    float64
10  symmetry_mean                         569 non-null    float64
11  fractal_dimension_mean                569 non-null    float64
12  radius_se                             569 non-null    float64
13  texture_se                             569 non-null    float64
14  perimeter_se                           569 non-null    float64
15  area_se                               569 non-null    float64
16  smoothness_se                         569 non-null    float64
17  compactness_se                        569 non-null    float64
18  concavity_se                          569 non-null    float64
19  concave points_se                     569 non-null    float64
20  symmetry_se                           569 non-null    float64
21  fractal_dimension_se                  569 non-null    float64
22  radius_worst                          569 non-null    float64
23  texture_worst                         569 non-null    float64
24  perimeter_worst                       569 non-null    float64
25  area_worst                            569 non-null    float64
26  smoothness_worst                     569 non-null    float64
27  compactness_worst                     569 non-null    float64
28  concavity_worst                       569 non-null    float64
29  concave points_worst                  569 non-null    float64
30  symmetry_worst                        569 non-null    float64
31  fractal_dimension_worst               569 non-null    float64
dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB
```

```
# Let's suppose we have multiple columns which are having the null values then how
# can we handle the multiple columns null values ?
# Separate out the categorical and numerical column from the dataset by using
# df.select_dtypes(include='datatype').column

# Iterate over the each of columns separately and then impute then
#
cat_column = df.select_dtypes(include='object').columns
num_cols = df.select_dtypes(exclude='object').columns
for col in num_cols:
    df[col].fillna(df[col].mean(), inplace=True)
```

Start coding or [generate](#) with AI.

```
Index(['diagnosis'], dtype='object')
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

/tmp/ipykernel_28587/4251004771.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chain. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

```
df[col].fillna(df[col].mean(), inplace=True)
```

```
df[num_cols].isna().sum()
```

	0
id	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
area_worst	0
smoothness_worst	0
compactness_worst	0
concavity_worst	0
concave points_worst	0
symmetry_worst	0
fractal_dimension_worst	0

```
df.drop('id', axis=1, inplace=True)
```

```
df
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30010	0.14710
1	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08690	0.07997
2	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19740	0.12730
3	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24140	0.10424
4	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19800	0.10434
...	...	...	...	...	...	...	...	...	...
564	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13270
565	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09747
566	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09251	0.09747
567	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15781
568	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000

569 rows × 10 columns

df.shape

(15, 8)

df.sample(3)

	Student_ID	Name	Age	Gender	Major	Maths	Physics	Chemistry
5	106	Frank	23	Male	Computer Science	95.0	88.0	82.0
12	113	Mia	20	NaN	Physics	99.0	NaN	96.0
3	104	David	22	Male	Biology	NaN	70.0	80.0

df['Maths'].fillna(df['Maths'].mean(), inplace=True)

```
# Download a dataset from the kaggle ->House Price Prediction
# Check how many columns are present into that
# drop id column from the dataframe
# and also drop the columns which are having null values more 50% of the total entries
# Impute the null values by using the mean, median & mode..
# In case of numerical columns you just have to check if there outlier present into that
# column or not ? -> impute with the median, otherwise mean impute..
```

df

df.isna().sum()

	0
Student_ID	0
Name	0
Age	0
Gender	0
Major	0
Maths	0
Physics	1
Chemistry	1

dtype: int64

```
data = {
    'col1':[1,2,3,4, np.nan,6]
```

```
}
df = pd.DataFrame(data)
df.fillna(method='bfill')
```

/tmp/ipykernel_28587/3487074322.py:5: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version.
df.fillna(method='bfill')

```
col1
0    1.0
1    2.0
2    3.0
3    4.0
4    6.0
5    6.0
```

```
# handling the null values ->
# We check in which column there is the null value present
# if the column is categorical column. -> then we must have to impute that column by using
# mode..
# if the column is numerical then we must impute with mean or median..But there is case
# that we can't use everywhere mean..because the outliers(extreme values) may affect
# the new value
# So, there we can use the median for that null value imputation...
```

```
# When should we drop the null values from the data frame ?
# If the number of null values is very less and that is not containing much information..
# only after discussing the team then we can drop that null values rows..
```

Start coding or [generate](#) with AI.

```
df['Gender'].unique()
```

```
array(['Female', nan, 'Male'], dtype=object)
```

```
mode = df['Gender'].mode()[0]
df['Gender'].fillna(mode, inplace=True)
```

/tmp/ipykernel_28587/1768366315.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chainable assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting the values is a copy.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting the values is a copy.
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df

```
df['Gender'].fillna(mode, inplace=True)
```

```
df['Major'].value_counts()
```

```
count
Major
Computer Science    3
Mathematics         3
Physics             3
Biology             3
Chemistry           3
```

```
dtype: int64
```

```
df['Major'].mode()[0]
```

```
'Biology'
```

```
df.isna().sum()
```

```
0
Student_ID 0
Name 0
Age 0
Gender 2
Major 0
Maths 0
Physics 1
Chemistry 1
```

dtype: int64

```
df['Chemistry'].fillna(df['Chemistry'].mean(), inplace=True)
```

/tmp/ipykernel_15819/1570172792.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chainable assignment. This behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting the values is a copy. To avoid this warning in pandas, you can explicitly use `df = df.copy()` before calling the `fillna` method. For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` or `df[col] = df[col].method(value, inplace=True)`.

```
df['Chemistry'].fillna(df['Chemistry'].mean(), inplace=True)
```

df

	Student_ID	Name	Age	Gender	Major	Maths	Physics	Chemistry
0	101	Alice	20	Female	Computer Science	88.0	85.0	90.0
1	102	Bob	21	Male	Mathematics	92.0	80.0	78.0
2	103	Charlie	19	Male	Physics	78.0	95.0	NaN
3	104	David	22	Male	Biology	85.0	70.0	80.0
4	105	Eva	20	Female	Chemistry	80.0	82.0	94.0
5	106	Frank	23	Male	Computer Science	95.0	88.0	82.0
6	107	Grace	21	Female	Mathematics	85.0	79.0	85.0
7	108	Henry	22	Male	Physics	74.0	91.0	88.0
8	109	Ivy	20	Female	Biology	70.0	75.0	83.0
9	110	Jack	19	Male	Chemistry	85.0	66.0	76.0
10	111	Karl	21	Male	Computer Science	91.0	84.0	89.0
11	112	Liam	22	Male	Mathematics	85.0	87.0	80.0
12	113	Mia	20	Female	Physics	99.0	NaN	96.0
13	114	Noah	21	Male	Biology	85.0	72.0	74.0
14	115	Olivia	19	Female	Chemistry	83.0	81.0	92.0

Start coding or [generate](#) with AI.

```
df['Gender'].isna().sum()
```

```
np.int64(0)
```

```
df['Gender'].mode()[0]
```

```
'Male'
```

```
df['Gender'].value_counts()
```

```

count
Gender
Male      10
Female     5
dtype: int64

```

```

# We impute the null values of the categorical columns by using most frequent data
# of that particular column..

```

```

mode_value = df['Gender'].mode()[0]
df['Gender'].fillna(mode_value, inplace=True)

```

```
df.isna().sum()
```

```

0
Student_ID  0
Name        0
Age         0
Gender      0
Major       0
Maths       0
Physics     0
Chemistry   0
dtype: int64

```

```
df.apply(lambda row : row['Maths'] + row['Chemistry']+row['Physics'], axis=1)
```

```

0
0  263.0
1  250.0
2   NaN
3   NaN
4  256.0
5  265.0
6   NaN
7  253.0
8  228.0
9   NaN
10 264.0
11 252.0
12  NaN
13  NaN
14 256.0
dtype: float64

```

```
import pandas as pd
```

```

data = {
    "Name": ["Vikash", "Amit", "Rahul", "Priya", "Neha", "Amit"],
    "Department": ["IT", "HR", "IT", "Finance", "HR", "HR"],
    "Marks": [85, 45, 70, 90, 67, 45],
    "Salary": [60000, 40000, 55000, 80000, 50000, 40000],
}

```

```
"Joining_Date": [
    "2024-01-15",
    "2023-06-20",
    "2022-11-05",
    "2024-03-10",
    "2023-08-25",
    "2023-06-20"
]
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)
```

Original DataFrame:

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
1	Amit	HR	45	40000	2023-06-20
2	Rahul	IT	70	55000	2022-11-05
3	Priya	Finance	90	80000	2024-03-10
4	Neha	HR	67	50000	2023-08-25
5	Amit	HR	45	40000	2023-06-20

```
df[(df['Department'] == 'IT') & (df['Salary'] > 50000)]
```

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
2	Rahul	IT	72	55000	2022-11-05

```
df[df['Marks'] > 70]
```

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
2	Rahul	IT	72	55000	2022-11-05
3	Priya	Finance	91	80000	2024-03-10

Start coding or [generate](#) with AI.

```
print("\n1. Single Condition (Marks > 70)")
print(df[df["Marks"] > 70])
```

1. Single Condition (Marks > 70)

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
2	Rahul	IT	72	55000	2022-11-05
3	Priya	Finance	91	80000	2024-03-10

```
print("\n2. Multiple Conditions with & (Marks > 70 and Salary > 55000)")
print(df[(df["Marks"] > 70) & (df["Salary"] > 55000)])
```

2. Multiple Conditions with & (Marks > 70 and Salary > 55000)

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
3	Priya	Finance	91	80000	2024-03-10

```
print("\nMultiple Conditions with | (Department is HR or Finance)")
print(df[(df["Department"] == "HR") | (df["Department"] == "Finance")])
```

Multiple Conditions with | (Department is HR or Finance)

	Name	Department	Marks	Salary	Joining_Date
1	Amit	HR	45	40000	2023-06-20
3	Priya	Finance	91	80000	2024-03-10
4	Neha	HR	67	50000	2023-08-25
5	Amit	HR	45	40000	2023-06-20

```
print("\nUsing ~ (NOT HR Department)")
print(df[~(df["Department"] == "HR")])
```

Using ~ (NOT HR Department)

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
2	Rahul	IT	72	55000	2022-11-05
3	Priya	Finance	91	80000	2024-03-10

```
df[df['Department'].isin(['IT', "HR"])]
```

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
1	Amit	HR	45	40000	2023-06-20
2	Rahul	IT	72	55000	2022-11-05
4	Neha	HR	67	50000	2023-08-25
5	Amit	HR	45	40000	2023-06-20

```
print("\n3. isin()")
print(df[df["Department"].isin(["IT", "Finance"])])
```

3. isin()

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
2	Rahul	IT	72	55000	2022-11-05
3	Priya	Finance	91	80000	2024-03-10

```
df[df['Marks'].between(60, 90)]
```

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
2	Rahul	IT	70	55000	2022-11-05
3	Priya	Finance	90	80000	2024-03-10
4	Neha	HR	67	50000	2023-08-25

```
print("\n4. between()")
print(df[df["Marks"].between(60, 90)])
```

4. between()

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
2	Rahul	IT	72	55000	2022-11-05
4	Neha	HR	67	50000	2023-08-25

```
df.sort_values('Salary', ascending=True).head(2)
```

	Name	Department	Marks	Salary	Joining_Date
1	Amit	HR	45	40000	2023-06-20
5	Amit	HR	45	40000	2023-06-20

```
print("\n5. sort_values() by Marks (Descending)")
print(df.sort_values("Marks", ascending=False))
```

5. sort_values() by Marks (Descending)

	Name	Department	Marks	Salary	Joining_Date
3	Priya	Finance	91	80000	2024-03-10
0	Vikash	IT	85	60000	2024-01-15
2	Rahul	IT	72	55000	2022-11-05
4	Neha	HR	67	50000	2023-08-25
1	Amit	HR	45	40000	2023-06-20
5	Amit	HR	45	40000	2023-06-20

df

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
1	Amit	HR	45	40000	2023-06-20
2	Rahul	IT	70	55000	2022-11-05
3	Priya	Finance	90	80000	2024-03-10
4	Neha	HR	67	50000	2023-08-25
5	Amit	HR	45	40000	2023-06-20

df.sample(frac=1).sort_index()

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
1	Amit	HR	45	40000	2023-06-20
2	Rahul	IT	70	55000	2022-11-05
3	Priya	Finance	90	80000	2024-03-10
4	Neha	HR	67	50000	2023-08-25
5	Amit	HR	45	40000	2023-06-20

```
print("\n6. sort_index()")
shuffled = df.sample(frac=1)
shuffled
print(shuffled.sort_index())
```

6. sort_index()

	Name	Department	Marks	Salary	Joining_Date
0	Vikash	IT	85	60000	2024-01-15
1	Amit	HR	45	40000	2023-06-20
2	Rahul	IT	72	55000	2022-11-05
3	Priya	Finance	91	80000	2024-03-10
4	Neha	HR	67	50000	2023-08-25
5	Amit	HR	45	40000	2023-06-20

```
print("\n7. groupby() - Group by Department")
grouped = df.groupby("Department")
print(grouped)
```

7. groupby() - Group by Department
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7d7044552150>

df.groupby("Department")['Salary'].mean()

Salary	
Department	
Finance	80000.000000
HR	43333.333333
IT	57500.000000

dtype: float64

```
print("\n8. Aggregations - Average Marks by Department")
print(df.groupby("Department")["Marks"].mean())
```

8. Aggregations - Average Marks by Department

Department	
Finance	91.000000
HR	52.333333
IT	78.500000

Name: Marks, dtype: float64

```
print("\nMultiple Aggregations")
print(
    df.groupby("Department").agg({
        "Marks": ["mean", "max", "min"],
        "Salary": ["sum", "mean", "max"]
    })
)
```

```
Multiple Aggregations
```

Department	Marks			Salary		
	mean	max	min	sum	mean	max
Finance	90.000000	90	90	80000	80000.000000	80000
HR	52.333333	67	45	130000	43333.333333	50000
IT	77.500000	85	70	115000	57500.000000	60000

```
df1 = pd.DataFrame({
    "Name": ["A", "B"],
    "Marks": [80, 90]
})
```

```
df2 = pd.DataFrame({
    "Name": ["C", "D"],
    "Marks": [70, 85]
})
```

df1

	Name	Marks
0	A	80
1	B	90

df2

	Name	Marks
0	C	70
1	D	85

```
print("Vertical Concatenation:")
print(pd.concat([df1, df2], ignore_index=True))
```

```
Vertical Concatenation:
```

	Name	Marks
0	A	80
1	B	90
2	C	70
3	D	85

```
print("Horizontal Concatenation:")
print(pd.concat([df1, df2], axis=1))
```

```
Horizontal Concatenation:
```

	Name	Marks	Name	Marks
0	A	80	C	70
1	B	90	D	85

```
df['Joining_Date']
```

Joining_Date	
0	2024-01-15
1	2023-06-20
2	2022-11-05
3	2024-03-10
4	2023-08-25
5	2023-06-20

dtype: object

```
print("\n10. pd.to_datetime()")
df["Joining_Date"] = pd.to_datetime(df["Joining_Date"])
print(df["Joining_Date"])
```

```
10. pd.to_datetime()
0    2024-01-15
1    2023-06-20
2    2022-11-05
3    2024-03-10
4    2023-08-25
5    2023-06-20
Name: Joining_Date, dtype: datetime64[ns]
```

```
df['Joining_Date'].dt.year
```

Joining_Date	
0	2024
1	2023
2	2022
3	2024
4	2023
5	2023

dtype: int32

```
print("\n11. Extract Date Components")
df["Year"] = df["Joining_Date"].dt.year
df["Month"] = df["Joining_Date"].dt.month
df["Day"] = df["Joining_Date"].dt.day
df["Weekday"] = df["Joining_Date"].dt.day_name()
```

11. Extract Date Components

df

	Name	Department	Marks	Salary	Joining_Date	Year	Month	Day	Weekday
0	Vikash	IT	85	60000	2024-01-15	2024	1	15	Monday
1	Amit	HR	45	40000	2023-06-20	2023	6	20	Tuesday
2	Rahul	IT	70	55000	2022-11-05	2022	11	5	Saturday
3	Priya	Finance	90	80000	2024-03-10	2024	3	10	Sunday
4	Neha	HR	67	50000	2023-08-25	2023	8	25	Friday
5	Amit	HR	45	40000	2023-06-20	2023	6	20	Tuesday

df

	Name	Department	Marks	Salary	Joining_Date	Year	Month	Day	Weekday
0	Vikash	IT	85	60000	2024-01-15	2024	1	15	Monday
1	Amit	HR	45	40000	2023-06-20	2023	6	20	Tuesday
2	Rahul	IT	70	55000	2022-11-05	2022	11	5	Saturday
3	Priya	Finance	90	80000	2024-03-10	2024	3	10	Sunday
4	Neha	HR	67	50000	2023-08-25	2023	8	25	Friday
5	Amit	HR	45	40000	2023-06-20	2023	6	20	Tuesday

```
df.duplicated()
```